System Design & Architecture Masterclass

The Definitive Master Guide: Web Hosting, Content Protection & Post-by-Post LinkedIn Engine

---

PART 1: Zero-Friction Hosting & Content Protection

Before posting on LinkedIn, you must host your masterclass so that your live URL is ready to share in the comments.

Step 1: Push Local Files to GitHub

Open terminal in your project root directory (f:\0. bank\0. work\sys design) and run:

```
git init
git add .
git commit -m 'feat: complete system design masterclass (616 topics, 108 SVGs)'
git branch -M main
git remote add origin https://github.com/Mr-Elegant/System-Design-Masterclass.git
git push -u origin main
```

Step 2: Enable GitHub Pages (Free Worldwide Hosting)

1. Go to your repository on GitHub: https://github.com/Mr-Elegant/System-Design-Masterclass.
2. Click **Settings** (top bar) → **Pages** (left sidebar).
3. Under **Build and deployment** → **Branch**, select **main** and **/ (root)**.
4. Click **Save**. Within 60 seconds, your site is live at: **https://mr-elegant.github.io/System-Design-Masterclass/**

Step 3: Built-In Content Protection & Anti-Theft

All 55 HTML files in your workspace already include active client-side protection:

- **Context Menu Lock:** Disables right-click to prevent 'Save As' and element inspection.
- **DevTools Interception:** Blocks F12, Ctrl+Shift+I, Ctrl+Shift+J, Ctrl+U (view source), and Ctrl+S.
- **Selective Copy Lock:** Protects curriculum text from bulk scraping while allowing users to copy code snippets.

PART 2: The LinkedIn Growth Strategy & Algorithm Rules

To get 10,000+ views, maximum saves, and recruiter/peer engagement, follow these 4 golden rules:

1. **Never Put External Links in the Post Body:** Putting a URL in your post body drops reach by 60-70%. Always write: *'■ Link in the first comment below! ■'* and comment immediately.
2. **Attach Animated GIFs:** Autoplays in followers' feeds, instantly boosting dwell time.
3. **First 60-Minute Rule:** Reply to every comment within 15-30 minutes to trigger the recommendation algorithm.
4. **Posting Cadence:** Post 3 times per week (Tuesday, Thursday, Saturday) between 7:30 AM – 9:00 AM.

PART 3: Post-by-Post Execution Catalog (Organized by Vault Folders)

All posts and matching animated diagrams are organized into: f:\0. bank\0. work\sys design\LinkedIn_Posting_Vault/.

■ Series 1: Grokking System Design Fundamentals (18 Posts)

Folder: LinkedIn_Posting_Vault/01_System_Design_Fundamentals_Series/

Post #01: Scalability, Latency vs Throughput & High Availability SLAs

Folder: 01_System_Design_Fundamentals_Series/Post_01_Scalability,_Latency_vs_Throughput_/ | **Attach:** animated_diagram.gif

```
■ Latency vs Throughput: The most common confusion in System Design Interviews:

Most engineers think making a system faster (lower latency) automatically increases throughput. But in distributed
systems, they are two completely different dimensions:

■■ Latency: The time taken to process a single request (e.g. 50ms P99 response time).
■ Throughput: The number of requests processed per second (e.g. 100,000 QPS).

■ The Classic Pipeline Tradeoff:
• Batching requests (e.g. waiting 10ms to group 100 events) increases Throughput by 5x, but slightly increases
individual Latency!
• Streaming requests one-by-one gives minimum Latency, but reduces overall Throughput due to network framing overhead.

■ Availability "The 9's" Quick Sizing:
• 99.9% (Three 9s) = 8.76 hours downtime/year.
• 99.99% (Four 9s - Standard FAANG SLA) = 52.6 minutes downtime/year.
• 99.999% (Five 9s - Mission Critical) = 5.26 minutes downtime/year!

■ How do you measure latency in production: P90, P95, or P99?

(■ Full live interactive guide in first comment below! ■)
#SystemDesign #Latency #Throughput #DistributedSystems #Performance #SoftwareEngineering

--- FIRST COMMENT ---
■ Live interactive masterclass & code: https://mr-elegant.github.io/System-Design-Masterclass/
```

Post #02: Layer 4 vs Layer 7 Load Balancing & Weighted Round-Robin

Folder: 01_System_Design_Fundamentals_Series/Post_02_Layer_4_vs_Layer_7_Load_Balancing_a/ | **Attach:** animated_diagram.gif

```
■■ Layer 4 (Transport) vs Layer 7 (Application) Load Balancing: Which one should you pick?

Load balancers are the gatekeepers of scalable systems. Choosing the wrong tier can cause severe packet bottlenecking:

■ Layer 4 Load Balancing (TCP/UDP):
• Operates at IP & Port level without decrypting TLS or reading HTTP payloads.
• Ultra-high throughput (millions of packets/sec via Linux IPVS / Maglev).
• Limitation: Cannot inspect HTTP headers, cookies, or route by URL path!

■ Layer 7 Load Balancing (HTTP/HTTPS/gRPC):
• Decrypts TLS and parses headers, cookies, and JSON payloads.
• Smart routing: `/api/v1/checkout` -> Payment Cluster; `/api/v1/search` -> Search Cluster.
• Features: Header-based rate limiting, JWT validation, WebSocket session affinity.
• Tradeoff: Higher CPU utilization due to SSL termination and packet parsing.

■ Production Best Practice:
A two-tier setup: Layer 4 ECMP/Maglev LB at the network edge for raw packet distribution -> Fans out to a fleet of
Layer 7 Envoy/NGINX gateways for application routing!

■ What load balancer do you run in production: Envoy, NGINX, HAProxy, or Cloud Load Balancers?

(■ Full live interactive guide in first comment below! ■)
#LoadBalancing #Networking #DevOps #SystemDesign #Infrastructure #Microservices

--- FIRST COMMENT ---
■ Live interactive masterclass & code: https://mr-elegant.github.io/System-Design-Masterclass/
```

Post #03: API Gateway Architecture: Rate Limiting, Auth & Circuit Breaking

Folder: 01_System_Design_Fundamentals_Series/Post_03_API_Gateway_Architecture/ | **Attach:** animated_diagram.gif

```
■ Why you should NEVER expose microservices directly to the public internet:

Without an API Gateway, every mobile and web client must talk directly to dozens of internal microservices, causing:
■ Tight coupling between client and internal service endpoints.
```

■ Duplicated authentication, SSL certificates, and logging logic in every service.
■ Vulnerability to DDoS attacks and noisy-neighbor resource starvation.

■ The API Gateway Pattern solves this by acting as a unified entry point:
1■■ Authentication & Authorization: Validates JWT / OAuth2 tokens at the edge and passes sanitized user identity headers (`X-User-Id: 42`) to downstream services.
2■■ Protocol Translation: Translates public REST/JSON requests into internal high-speed gRPC/Protobuf calls.
3■■ Rate Limiting & WAF: Enforces Token Bucket / Sliding Window limits per API key before traffic reaches backend workers.
4■■ SSL Termination & Circuit Breaking: Absorbs TLS encryption overhead and prevents cascading backend failures.

■ Do you use an off-the-shelf gateway (Kong, Envoy, AWS API Gateway) or build custom in Node/Go?

(■ Full live interactive guide in first comment below! ■)
#APIGateway #Microservices #Security #Envoy #BackendEngineering #SystemDesign

--- FIRST COMMENT ---
■ Live interactive masterclass & code: https://mr-elegant.github.io/System-Design-Masterclass/

Post #04: HTTP/1.1 vs HTTP/2 vs HTTP/3 (QUIC) vs WebSockets

Folder: 01_System_Design_Fundamentals_Series/Post_04_HTTP_1.1_vs_HTTP_2_vs_HTTP_3_(QUIC)/ | **Attach:** animated_diagram.gif

■ The evolution of web protocols: Why HTTP/3 moves from TCP to UDP (QUIC):

Understanding networking protocols is essential for optimizing client-server latency:

1■■ HTTP/1.1: Head-of-Line (HoL) Blocking
• Only 1 request/response per TCP connection at a time. Browsers open 6 parallel TCP connections to compensate, causing connection churn.

2■■ HTTP/2: Binary Framing & Multiplexing
• Multiplexes dozens of concurrent streams over a single TCP connection.
• Limitation: TCP-level packet loss stalls ALL multiplexed streams (TCP Head-of-Line blocking).

3■■ HTTP/3 over QUIC (UDP):
• Runs over UDP with user-space congestion control.
• Solves TCP HoL blocking: Packet loss on Stream A does NOT block Stream B!
• Zero-RTT Handshake: Re-establishes encrypted connections in 0 milliseconds using cached connection tokens.

4■■ WebSockets:
• Full-duplex persistent bidirectional TCP connection with minimal 2-byte framing overhead (ideal for multiplayer games, chat, Figma canvas sync).

■ Is your production API running HTTP/2 or HTTP/3 yet?

(■ Full live interactive guide in first comment below! ■)
#Networking #HTTP3 #QUIC #WebSockets #Performance #SystemDesign

--- FIRST COMMENT ---
■ Live interactive masterclass & code: https://mr-elegant.github.io/System-Design-Masterclass/

Post #05: DNS Anycast Routing, GeoDNS & Latency Optimization

Folder: 01_System_Design_Fundamentals_Series/Post_05_DNS_Anycast_Routing,_GeoDNS_and_Lat/ | **Attach:** animated_diagram.gif

■ How Geo-DNS & BGP Anycast route users to the closest data center in < 20ms:

When a user types `google.com`, how does the internet ensure they connect to a server 50 miles away rather than across the ocean?

■ Two Core Routing Mechanisms:

1■■ GeoDNS (Latency-Based Routing):
• The DNS resolver looks up the client's recursive DNS IP.
• Returns the IP address of the geographically closest data center.
• Limitation: Relies on recursive resolver location (which may differ from actual user IP if using 8.8.8.8 without EDNS-Client-Subnet).

2■■ BGP Anycast Routing (Cloudflare, AWS Route53):
• Multiple physical data centers across 300+ global cities advertise the EXACT SAME IP address via Border Gateway Protocol (BGP).
• Internet routers automatically steer user packets along the shortest physical AS-path to the nearest edge point of presence (PoP)!

```
■ What DNS provider do you rely on: AWS Route53, Cloudflare, or NS1?

(■ Full live interactive guide in first comment below! ■)
#DNS #Networking #Anycast #Cloudflare #Infrastructure #SystemDesign

--- FIRST COMMENT ---
■ Live interactive masterclass & code: https://mr-elegant.github.io/System-Design-Masterclass/
```

Post #06: Caching Strategies: Cache-Aside vs Write-Through vs Write-Behind

Folder: 01_System_Design_Fundamentals_Series/Post_06_Caching_Strategies/ | **Attach:** animated_diagram.gif

```
■ Cache-Aside, Write-Through, Write-Behind: How to pick the right caching topology:

Caching is the #1 way to achieve sub-millisecond read latency, but picking the wrong write strategy can corrupt your
data!

1■■ Cache-Aside (Lazy Loading):
• Read: App queries Cache. On Miss, reads DB and populates Cache.
• Write: App writes to DB, then evicts/invalidates cache key.
• Best for: General read-heavy systems (e.g. user profiles).

2■■ Write-Through:
• App writes to Cache; Cache synchronously writes to DB before confirming.
• Strength: Cache is never stale.
• Weakness: Double-write latency penalty on every update.

3■■ Write-Behind (Write-Back):
• App writes to Cache; Cache immediately confirms and asynchronously batches disk flushes!
• Strength: Ultra-fast write throughput (absorbs write bursts).
• Risk: If the cache crashes before flushing to disk, recent writes are LOST!

4■■ Eviction Policies:
• LRU (Least Recently Used), LFU (Least Frequently Used), FIFO, and 2Q.

■ What is your go-to cache eviction policy in Redis?

(■ Full live interactive guide in first comment below! ■)
#Caching #Redis #Performance #SystemDesign #SoftwareEngineering #Backend

--- FIRST COMMENT ---
■ Live interactive masterclass & code: https://mr-elegant.github.io/System-Design-Masterclass/
```

Post #07: Database Sharding: Range vs Hash vs Directory-Based Partitioning

Folder: 01_System_Design_Fundamentals_Series/Post_07_Database_Sharding/ | **Attach:** animated_diagram.gif

```
■■ When your database exceeds 5TB and 50,000 QPS: How to shard like a FAANG architect:

Vertical scaling hits a physical hardware ceiling. Horizontal sharding splits large datasets across multiple
independent database servers.

■ 3 Sharding Strategies Compared:

1■■ Range-Based Sharding:
• Keys partitioned by value ranges (e.g., User IDs 1-1M -> Shard 1; 1M-2M -> Shard 2).
• Strength: Easy range queries (`BETWEEN 100 AND 500`).
• Weakness: Severe Hotspotting! New active users all hammer the highest shard.

2■■ Hash-Based Sharding (Consistent Hashing):
• `Shard = Hash(UserID) % N_Shards`.
• Strength: Perfectly uniform data distribution.
• Weakness: Range queries require querying ALL shards (Scatter-Gather).

3■■ Directory-Based Sharding:
• A centralized lookup service maps entity IDs to specific shard locations.
• Strength: Flexible dynamic rebalancing and custom customer tenant routing.
• Weakness: Lookup service becomes a single point of failure (SPOF) if not heavily cached.

■ What sharding key do you pick for multi-tenant SaaS: OrganizationId or UserId?

(■ Full live interactive guide in first comment below! ■)
#Sharding #Databases #PostgreSQL #Scalability #SystemDesign #Architecture

--- FIRST COMMENT ---
```

```
■ Live interactive masterclass & code: https://mr-elegant.github.io/System-Design-Masterclass/
```

Post #08: Forward Proxy vs Reverse Proxy vs VPN: The Structural Differences

Folder: 01_System_Design_Fundamentals_Series/Post_08_Forward_Proxy_vs_Reverse_Proxy_vs_V/ | **Attach:** animated_diagram.gif

```
■■ Forward Proxy vs Reverse Proxy vs VPN: Clear architectural breakdown:

Engineers often mix up Proxies and VPNs. Here is the definitive difference:

1■■ Forward Proxy (Protects the Client):
• Sits in front of client devices (e.g. corporate office).
• Intercepts outbound traffic to block malicious sites, cache external content, and mask employee IP addresses.
• Server sees the Proxy's IP, NOT the client's.

2■■ Reverse Proxy (Protects the Server):
• Sits in front of backend servers (e.g. NGINX, HAProxy).
• Intercepts inbound traffic from the internet to load balance, terminate SSL, and cache responses.
• Client sees the Proxy's IP, NOT the internal backend servers!

3■■ Virtual Private Network (VPN):
• Encrypts ALL device network traffic at the OS network interface level (Layer 3/4) through a secure tunnel
(WireGuard/IPSec) into a private network.

■ Do you use NGINX, Envoy, or Caddy as your reverse proxy of choice?

(■ Full live interactive guide in first comment below! ■)
#Networking #Security #NGINX #Proxy #DevOps #SystemDesign

--- FIRST COMMENT ---
■ Live interactive masterclass & code: https://mr-elegant.github.io/System-Design-Masterclass/
```

Post #09: Database Replication: Active-Passive vs Active-Active & Replication Lag

Folder: 01_System_Design_Fundamentals_Series/Post_09_Database_Replication/ | **Attach:** animated_diagram.gif

```
■ Master-Replica vs Multi-Master Replication: Mitigating the dreaded Replication Lag:

Replication ensures data durability and high availability, but async replication introduces consistency traps:

1■■ Active-Passive (Primary-Replica):
• 1 Master handles all Writes. Multiple Replicas handle Reads.
• Synchronous Replication: Master waits for replica confirmation (zero data loss, but slow writes).
• Asynchronous Replication: Master commits locally immediately (ultra-fast writes, but risk of data loss on failover).

2■■ Active-Active (Multi-Master):
• Multiple nodes accept both Reads and Writes.
• Requires Conflict Resolution (Last-Write-Wins timestamps, Vector Clocks, or CRDTs).

■■ The "Read-Your-Own-Writes" Consistency Trap:
User updates their profile picture (writes to Master) -> Immediately refreshes feed (reads from lagged Replica) -> Old
picture is shown!
■ Solution: Route reads for the updating user to Master for 5 seconds, or verify replica replication offset before
serving!

■ How do you handle read-after-write consistency in your systems?

(■ Full live interactive guide in first comment below! ■)
#Databases #Replication #PostgreSQL #HighAvailability #SystemDesign

--- FIRST COMMENT ---
■ Live interactive masterclass & code: https://mr-elegant.github.io/System-Design-Masterclass/
```

Post #10: CAP Theorem & PACELC: The Mathematical Tradeoffs of Distributed Systems

Folder: 01_System_Design_Fundamentals_Series/Post_10_CAP_Theorem_and_PACELC/ | **Attach:** animated_diagram.gif

```
■■ The mathematical reality of distributed systems: Why you can never have CA over wide-area networks:

Network cables get cut, routers reboot, and datacenters experience packet loss. In a distributed network, Network
Partition (P) is an unavoidable physical reality!

■ Therefore, your choice during a partition is strictly binary:

■ CP (Consistency + Partition Tolerance):
• Reject writes if quorum cannot be reached to prevent stale reads.
```

```
• Examples: Google Spanner, HBase, ZooKeeper, etcd.
• Use case: Banking balances, seat reservations, consensus state.

■ AP (Availability + Partition Tolerance):
• Accept writes on all reachable nodes, allowing divergent states to be merged later.
• Examples: Amazon DynamoDB, Apache Cassandra, CouchDB.
• Use case: Shopping carts, social media likes, telemetry logs.

■ PACELC Extension:
Even under normal operation (No partition), choose Latency (EL) or Consistency (EC)!

■ In your current architecture, did you pick CP or AP?

(■ Full live interactive guide in first comment below! ■)
#CAPTheorem #DistributedSystems #SoftwareEngineering #Databases #SystemDesign

--- FIRST COMMENT ---
■ Live interactive masterclass & code: https://mr-elegant.github.io/System-Design-Masterclass/
```

Post #11: B+ Trees vs LSM-Trees: How Database Storage Engines Work Under the Hood

Folder: 01_System_Design_Fundamentals_Series/Post_11_B+_Trees_vs_LSM-Trees/ | **Attach:** animated_diagram.gif

```
■ B+ Tree vs LSM-Tree: Why PostgreSQL uses B+ Trees while Cassandra/RocksDB uses LSM-Trees:

The fundamental tradeoff in database internals is Random Read Latency vs Sequential Write Throughput:

■ B+ Trees (RDBMS: PostgreSQL, MySQL InnoDB):
• Ordered balanced tree where all data pointers live in leaf pages linked horizontally.
• Strength: Ultra-fast point reads ($O(\log N)$) and range scans.
• Bottleneck: Writes require random disk I/O to update tree pages, causing write amplification.

■ LSM-Trees (Log-Structured Merge-Trees: Cassandra, RocksDB, ScyllaDB):
• Writes append sequentially to an in-memory `MemTable` (SkipList) and Write-Ahead Log (WAL).
• When full, flushes to disk as immutable `SSTables`.
• Background compaction merges SSTables and purges tombstones.
• Strength: Blazing-fast sequential write throughput ($O(1)$ append).
• Tradeoff: Reads must check MemTable + Bloom Filters + multiple SSTables.

■ If you are designing an IoT sensor ingestion platform, which storage engine do you choose?

(■ Full live interactive guide in first comment below! ■)
#Databases #PostgreSQL #Cassandra #DataStructures #SystemDesign #SoftwareArchitecture

--- FIRST COMMENT ---
■ Live interactive masterclass & code: https://mr-elegant.github.io/System-Design-Masterclass/
```

Post #12: Bloom Filters: $O(1)$ Probabilistic Space-Saving Data Structures

Folder: 01_System_Design_Fundamentals_Series/Post_12_Bloom_Filters/ | **Attach:** animated_diagram.gif

```
■ How Bloom Filters save Bigtable, Cassandra, and Redis from wasted disk lookups in $O(1)$ time:

How do you check if a username exists across 500,000,000 records without touching disk?

■ The Solution: Bloom Filters (A space-efficient probabilistic bit array).

1■■ The Mechanism:
• A bit array of size $m$ initialized to all 0s.
• Uses $k$ independent cryptographic hash functions.
• When adding an element: Set bits at indices $h_1(x), h_2(x), \dots, h_k(x)$ to 1.
• When querying: If ANY of the $k$ bits is 0 -> The element is **DEFINITIVELY NOT** in the set (0% False Negative)!
• If ALL $k$ bits are 1 -> The element is **PROBABLY** in the set (small, tunable False Positive probability).

2■■ The Math:
For 1 billion keys with 1% false positive rate: Requires only ~1.2 GB of RAM (less than 10 bits per key)!

3■■ Applications:
• Cassandra/RocksDB: Skips disk SSTable reads for non-existent keys.
• Web Crawlers: Avoids re-crawling URLs.
• Medium/Twitter: Avoids showing users recommended articles they already read.

■ Have you used Bloom Filters or Cuckoo Filters in your projects?

(■ Full live interactive guide in first comment below! ■)
```

```
#BloomFilters #Algorithms #Redis #DataStructures #SystemDesign #Performance

--- FIRST COMMENT ---
■ Live interactive masterclass & code: https://mr-elegant.github.io/System-Design-Masterclass/
```

Post #13: Distributed System Patterns: Heartbeats, Fencing Tokens & Write-Ahead Logs

Folder: 01_System_Design_Fundamentals_Series/Post_13_Distributed_System_Patterns/ | **Attach:** animated_diagram.gif

```
■■ 4 Battle-Tested Patterns for Building Bulletproof Distributed Systems:

Distributed systems are prone to partial failures, network splits, and GC pauses. Here are the 4 fundamental patterns
to survive them:

1■■ Write-Ahead Log (WAL):
• Append every transaction to an immutable disk log BEFORE updating in-memory state.
• Guarantees crash recovery and durability across sudden power outages.

2■■ Fencing Tokens (Preventing Split-Brain):
• Distributed locks can expire during long GC pauses.
• A monotonic incrementing fencing token ($T_1, T_2, T_3$) is issued with each lock.
• Storage rejects writes with older token numbers ($T < T_{current}$), preventing stale zombie masters from corrupting
state!

3■■ Heartbeat with Phi Accrual Failure Detector:
• Instead of a binary up/down threshold, calculates a continuous suspicion probability based on historical network
latency variance.

4■■ Idempotent Consumers:
• Every message carries a unique ID; consumers store processed IDs in Redis with atomic `SETNX`.

■ How do you handle zombie master split-brain in your clusters?

(■ Full live interactive guide in first comment below! ■)
#DistributedSystems #Architecture #Reliability #DevOps #SystemDesign #Backend

--- FIRST COMMENT ---
■ Live interactive masterclass & code: https://mr-elegant.github.io/System-Design-Masterclass/
```

Post #14: Zero Trust Security, TLS 1.3 Handshake & JWT Best Practices

Folder: 01_System_Design_Fundamentals_Series/Post_14_Zero_Trust_Security,_TLS_1.3_Handsh/ | **Attach:** animated_diagram.gif

```
■ Zero-Trust Architecture: Why perimeter firewalls are dead in modern cloud engineering:

The old security model assumed everything inside the corporate intranet was safe. Zero Trust assumes the internal
network is ALREADY compromised!

■ Core Zero-Trust Principles:

1■■ Mutual TLS (mTLS) Everywhere:
• Microservices don't just verify client identity; every East-West microservice connection validates cryptographic
X.509 certificates in both directions via Envoy sidecars.

2■■ TLS 1.3 1-RTT Handshake:
• Slashes connection latency by removing round trips: Establishes cipher negotiation and key exchange in a single
round trip (or 0-RTT resumption).

3■■ Stateless Short-Lived JWT + Refresh Token Rotation:
• Access tokens expire in 15 minutes (cryptographically verified locally via RS256 public key without hitting Auth
DB).
• Refresh tokens stored in Redis with one-time-use rotation (any reused refresh token revokes the entire user session
immediately!).

■ Do you use asymmetric RS256 or symmetric HS256 for your JWT signatures?

(■ Full live interactive guide in first comment below! ■)
#Security #ZeroTrust #JWT #OAuth #DevOps #SystemDesign #CyberSecurity

--- FIRST COMMENT ---
■ Live interactive masterclass & code: https://mr-elegant.github.io/System-Design-Masterclass/
```

Post #15: Message Brokers: Apache Kafka vs RabbitMQ vs AWS SQS

Folder: 01_System_Design_Fundamentals_Series/Post_15_Message_Brokers/ | **Attach:** animated_diagram.gif

```
■ Kafka vs RabbitMQ vs SQS: How to choose the right messaging backbone for your architecture:

Picking the wrong message broker can cause unmanageable queue backlogs or lost messages.

1■■ RabbitMQ (Smart Broker, Dumb Consumer):
• Routing: Rich AMQP routing exchanges (Direct, Fanout, Topic, Headers).
• Message Life: Deletes messages immediately upon consumer acknowledgment.
• Best for: Complex transactional routing, background tasks (Celery), and low-latency priority queues.

2■■ Apache Kafka (Dumb Broker, Smart Consumer):
• Model: Distributed, partitioned, immutable append-only commit log.
• Retention: Retains messages on disk for days/weeks. Consumers track their own read offsets.
• Replayability: Allows replaying historical event streams from any timestamp!
• Best for: High-throughput event streaming (1M+ events/sec), Change Data Capture (CDC), and real-time analytics.

3■■ AWS SQS:
• Model: Fully managed serverless queue.
• Best for: Cloud-native async task decoupling with zero infrastructure maintenance.

■ What is your primary queue technology: Kafka, RabbitMQ, SQS, or Redis Streams?

(■ Full live interactive guide in first comment below! ■)
#ApacheKafka #RabbitMQ #MessagingQueues #EventDriven #SystemDesign #Backend

--- FIRST COMMENT ---
■ Live interactive masterclass & code: https://mr-elegant.github.io/System-Design-Masterclass/
```

Post #16: Distributed Storage: Object Storage (S3) vs Block Storage (EBS) vs File Storage (NFS)

Folder: 01_System_Design_Fundamentals_Series/Post_16_Distributed_Storage/ | **Attach:** animated_diagram.gif

```
■ Object Storage (S3) vs Block Storage (EBS) vs File Storage (EFS): Structural differences:

Storage architecture dictates performance, latency, and cost at petabyte scale:

1■■ Block Storage (AWS EBS, SAN):
• Exposes raw storage blocks over high-speed networks (iSCSI, NVMe-oF).
• Can be formatted with any filesystem (ext4, XFS).
• Best for: Databases (PostgreSQL, MySQL) requiring sub-millisecond random I/O.
• Limitation: Typically single-instance attachment.

2■■ File Storage (NFS, AWS EFS):
• Hierarchical folder/file tree with POSIX permissions.
• Multi-attach: Hundreds of compute instances can mount and read/write concurrently.
• Best for: Shared application assets, legacy migrations, CMS file trees.

3■■ Object Storage (AWS S3, Ceph, MinIO):
• Flat namespace: Buckets and Keys with metadata headers.
• Accessible via REST APIs (`GET`, `PUT`, `DELETE`).
• Infinite horizontal scalability, 11 9's durability ($99.999999999\%$ via Reed-Solomon Erasure Coding).
• Best for: Media blobs, backups, video streaming, data lakes.

■ How much storage do your production applications manage: Gigabytes, Terabytes, or Petabytes?

(■ Full live interactive guide in first comment below! ■)
#CloudStorage #AWS #S3 #DistributedSystems #DevOps #SystemDesign

--- FIRST COMMENT ---
■ Live interactive masterclass & code: https://mr-elegant.github.io/System-Design-Masterclass/
```

Post #17: Monolith vs Microservices vs Serverless: The Evolutionary Roadmap

Folder: 01_System_Design_Fundamentals_Series/Post_17_Monolith_vs_Microservices_vs_Server/ | **Attach:** animated_diagram.gif

```
■■ Monolith vs Microservices vs Serverless: When to evolve your architecture:

Premature microservices adoption is the #1 killer of startup velocity. Here is the realistic evolutionary roadmap:

1■■ The Majestic Modular Monolith (Phase 1):
• Single deployable unit with strictly enforced domain boundaries (folders/modules).
• Single ACID database. Zero distributed transaction overhead.
• Best for: Startups, new products, small engineering teams (< 20 engineers).

2■■ Microservices (Phase 2):
• Decouples domain services with private databases when team size exceeds 50+ engineers to prevent merge conflicts and
deployment gridlocks.
```

```
• Requires investment in: Service Mesh, Distributed Tracing (OpenTelemetry), CI/CD pipelines, and Kafka event buses.

3■■ Serverless & Edge Compute (Phase 3):
• Event-driven ephemeral execution (AWS Lambda, Cloudflare Workers).
• Best for: Spiky unpredictable traffic, image processing, webhook handlers, and edge personalization.

■ Is your team working on a Monolith or Microservices architecture?

(■ Full live interactive guide in first comment below! ■)
#Architecture #Microservices #Monolith #Serverless #DevOps #SoftwareEngineering

--- FIRST COMMENT ---
■ Live interactive masterclass & code: https://mr-elegant.github.io/System-Design-Masterclass/
```

Post #18: AI & LLM System Fundamentals: PagedAttention, FlashAttention & KV-Cache

Folder: 01_System_Design_Fundamentals_Series/Post_18_AI_and_LLM_System_Fundamentals/ | **Attach:** animated_diagram.gif

```
■ The Hardware & System Architecture of LLM Inference (FlashAttention & PagedAttention):

Why is LLM generation so computationally expensive, and how do modern serving engines optimize it?

■ 3 Breakthrough Systems Innovations:

1■■ KV-Cache & Memory Bottleneck:
In Autoregressive LLM generation, past Key and Value attention tensors are cached in GPU VRAM to avoid recalculating
all tokens. At batch size 64 with 4k context: KV-Cache consumes 30GB+ of VRAM!

2■■ PagedAttention (vLLM):
Traditional allocators reserve contiguous VRAM for maximum context length, wasting 60-80% of GPU memory (internal
fragmentation). PagedAttention manages KV-cache using virtual memory pages, boosting throughput by 4x!

3■■ FlashAttention-3:
Tiling the Attention matrix to fit entirely into fast on-chip GPU SRAM (100 TB/s bandwidth), avoiding slow
High-Bandwidth Memory (HBM) round-trips!

4■■ Speculative Decoding:
A small 1B draft model generates 4 candidate tokens; a large 70B target model validates them in a single forward pass,
yielding a 2.5x to 3x latency speedup!

■ Have you optimized LLM inference in production using vLLM or TensorRT-LLM?

(■ Full live interactive guide in first comment below! ■)
#GenerativeAI #LLM #FlashAttention #vLLM #MachineLearning #SystemDesign

--- FIRST COMMENT ---
■ Live interactive masterclass & code: https://mr-elegant.github.io/System-Design-Masterclass/
```

■ Series 2: System Design Interview Flagships & Papers (8 Posts)

Folder: LinkedIn_Posting_Vault/02_System_Design_Interview_Flagships_Series/

SDI Post #01: The 45-Minute 7-Step FAANG Interview Framework

Folder: 02_System_Design_Interview_Flagships_Series/Post_01_Master_Architecture_Blueprint/ | **Attach:** animated_diagram.gif

```
■■ How Staff & Principal engineers structure the 45-minute System Design Interview to get L6/L7 offers:

Most candidates fail System Design Interviews not because they don't know distributed components, but because they
fail to pace the 45-minute window.

Here is the exact time allocation used by FAANG interviewers:

■ 00:00 - 05:00 | Step 1: Requirements Scope
Clarify 3-4 core Functional use cases & establish Non-Functional SLOs (Latency < 100ms P99, 99.99% Availability).

■ 05:00 - 10:00 | Step 2: Back-of-the-Envelope Math
DAU -> QPS (Read/Write ratio) -> Storage per year -> Network Bandwidth -> 80/20 RAM Cache rule.

■ 10:00 - 15:00 | Step 3: APIs & Data Model
Define REST/gRPC contracts with status codes and SQL/NoSQL schema DDL with partition keys.

■ 15:00 - 25:00 | Step 4: High-Level Architecture
Draw the baseline end-to-end data flow: Client -> CDN/LB -> API Gateway -> Services -> Cache -> DB -> Async Message
Bus.

■ 25:00 - 38:00 | Step 5 & 6: Component Deep Dives
Detailed algorithms (QuadTree, SkipList, Vector Clocks), concurrency control, and replication protocols.

■ 38:00 - 45:00 | Step 7: Trade-offs & Fault Tolerance
SPOF elimination, CAP theorem trade-offs, rate limiting, disaster recovery, and interviewer curveballs.

■ Which phase of the 45-minute SDI do you find most stressful? Let me know below!

(■ Full live interactive guide in first comment below! ■)
#SystemDesign #CodingInterview #FAANG #SoftwareEngineering #TechCareers #Architecture

--- FIRST COMMENT ---
■ Live interactive masterclass & code: https://mr-elegant.github.io/System-Design-Masterclass/
```

SDI Post #02: Consistent Hashing with Virtual Nodes & Minimum Disruption

Folder: 02_System_Design_Interview_Flagships_Series/Post_02_Amazon_Dynamo_VectorClocks/ | **Attach:** animated_diagram.gif

```
■ Why naive hash mod (hash(key) % N) breaks horizontal scaling in caching clusters:

When a cache cluster has N servers, using `hash(key) % N` means that when 1 server crashes or is added, almost ALL
keys (~100%) rehash to different servers, causing a catastrophic cache stampede on the primary database!

■ The Solution: Consistent Hashing on a $2^{128}$ Token Ring.

1■■ Ring Topology: Both cache servers and data keys hash onto the same circular ring space [0, 2^128 - 1].
2■■ Clockwise Routing: A key is assigned to the first server whose token is >= the key's hash.
3■■ Minimum Disruption: When a server is added or removed, ONLY keys between adjacent nodes move (average K/N keys
relocated instead of 100%).
4■■ Virtual Nodes: Assigning 150-200 virtual tokens per physical server reduces load distribution variance below
$\sigma \approx 1/\sqrt{V} \approx 7\%$.

(Full Python & TypeScript consistent hashing ring implementation in the guide!)

■ Have you used Consistent Hashing in Redis Cluster, Cassandra, or Memcached?

(■ Full live interactive guide in first comment below! ■)
#ConsistentHashing #Caching #DistributedSystems #Redis #SystemDesign #SoftwareArchitecture

--- FIRST COMMENT ---
■ Live interactive masterclass & code: https://mr-elegant.github.io/System-Design-Masterclass/
```

SDI Post #03: PACELC Theorem: Beyond the Simplistic CAP Theorem

Folder: 02_System_Design_Interview_Flagships_Series/Post_03_Apache_Kafka_ZeroCopy/ | **Attach:** animated_diagram.gif

```
■■ Why the classic CAP Theorem is incomplete for real-world distributed databases:

Every engineer knows CAP: In presence of Partition (P), choose Availability (A) or Consistency (C).
```

```
But what happens when the network is working normally (NO partition)?

■ That's why Daniel Abadi formulated the PACELC Theorem:
■ If Partition (P): choose Availability (A) OR Consistency (C).
■ ELSE (E): choose Latency (L) OR Consistency (C).

Real Database Classifications:
■ PA/EL (Amazon DynamoDB, Apache Cassandra):
During partition, stays Available (PA). Under normal operation, optimizes for Sub-10ms Latency (EL) using eventual consistency.
■ PC/EC (Google Spanner, CockroachDB):
During partition, preserves Consistency (PC). Under normal operation, preserves strict Consistency (EC) via multi-phase consensus (Raft/Paxos/TrueTime) at the cost of higher latency.
■ MongoDB: PC/EC by default, tunable to PA/EL.

■ Do your production databases prioritize Latency (EL) or Consistency (EC)?

(■ Full live interactive guide in first comment below! ■)
#Databases #CAPTheorem #DistributedSystems #NoSQL #SystemDesign #BackendEngineering

--- FIRST COMMENT ---
■ Live interactive masterclass & code: https://mr-elegant.github.io/System-Design-Masterclass/
```

SDI Post #04: REST vs gRPC vs WebSocket vs Server-Sent Events (SSE)

Folder: 02_System_Design_Interview_Flagships_Series/Post_04_Multimodal_ColPali_RAG/ | **Attach:** animated_diagram.gif

```
■ Choosing the right client-server communication protocol in distributed architectures:

Picking the wrong communication protocol can cause connection exhaustion, high CPU overhead, or head-of-line blocking. Here is the decision matrix:

1■■ REST over HTTP/1.1 or HTTP/2:
• Best for: Public APIs, CRUD endpoints, web clients.
• Strengths: Universally supported, human-readable JSON, built-in edge caching.
• Weaknesses: High payload overhead, text-based serialization.

2■■ gRPC over HTTP/2:
• Best for: Internal Microservices east-west communication.
• Strengths: Protocol Buffers binary serialization (up to 7x faster than JSON), bidirectional streaming, strict typed contracts.
• Weaknesses: Harder to inspect in browser dev tools without proxy (gRPC-Web).

3■■ WebSockets:
• Best for: True bidirectional real-time communication (multiplayer games, chat, Figma canvas sync).
• Strengths: Full-duplex persistent TCP connection with minimal 2-byte framing overhead.
• Weaknesses: Stateful connections require sticky load balancing and complex horizontal scaling.

4■■ Server-Sent Events (SSE):
• Best for: Unidirectional server-to-client streaming (ChatGPT token generation, stock tickers, notification feeds).
• Strengths: Runs over standard HTTP/2, built-in automatic client reconnection.

■ What protocol do you use for AI streaming: SSE or WebSockets?

(■ Full live interactive guide in first comment below! ■)
#API #gRPC #WebSockets #REST #Microservices #SystemDesign #SoftwareEngineering

--- FIRST COMMENT ---
■ Live interactive masterclass & code: https://mr-elegant.github.io/System-Design-Masterclass/
```

SDI Post #05: Designing TinyURL: Base62 vs Key Generation Service (KGS)

Folder: 02_System_Design_Interview_Flagships_Series/Post_05_ChatGPT_LLM_Streaming/ | **Attach:** animated_diagram.gif

```
■ Why MD5/SHA-256 hashing fails for TinyURL and how a Key Generation Service (KGS) fixes it:

Designing a URL Shortener seems simple until you hit 100,000,000 URLs/day and need zero collision guarantees.

■ The Flawed Approach: MD5(URL) -> Base62
Hashing a long URL gives 128 bits. Taking the first 7 characters causes hash collisions. Appending user IDs or random salts requires continuous database collision checks, causing high latency write bottlenecks!

■ The Staff-Level Solution: Dedicated Key Generation Service (KGS)
1■■ Pre-generate 7-character Base62 keys ($62^7 \approx 3.52 \text{ Trillion}$ unique keys).
2■■ Store keys in two tables: `Used_Keys` and `Available_Keys`.
```

```
3■■ KGS server loads 5,000 available keys into RAM.
4■■ When a request arrives, KGS hands out a key in < 1 millisecond with ZERO database hashing or collision checks!
5■■ Redundant KGS replicas with master-standby leases ensure no duplicate keys are ever handed out.

(Includes complete Python + TypeScript production code with Redis caching!)

■ How would you handle key recovery when shortened URLs expire?

(■ Full live interactive guide in first comment below! ■)
#TinyURL #SystemDesign #Algorithms #CodingInterview #DistributedSystems #SoftwareArchitecture

--- FIRST COMMENT ---
■ Live interactive masterclass & code: https://mr-elegant.github.io/System-Design-Masterclass/
```

SDI Post #06: Designing Uber Backend: H3 Hexagons & Dynamic Surge Multiplier

Folder: 02_System_Design_Interview_Flagships_Series/Post_06_Figma_Realtime_CRDT/ | **Attach:** animated_diagram.gif

```
■ How Uber matches riders to nearby drivers in < 2 seconds across 10,000+ global cities:

Traditional SQL spatial queries (`ST_DWithin` on latitude/longitude) degrade exponentially when ingesting 1,000,000
GPS updates every 4 seconds.

■ The Uber Architecture:
1■■ Uber H3 Hierarchical Spatial Index:
Partitions city terrain into uniform hexagons (Resolution 7 to 9). Unlike squares, hexagons have identical distances
to all 6 neighboring cells, preventing directional distortion!

2■■ Ingestion Pipeline:
Drivers stream GPS pings over persistent WebSockets -> Ingested into Redis Geospatial (`GEOADD`) with 30-second TTLs.

3■■ Dynamic Surge Pricing Engine:
Computes the supply-demand ratio per hexagon cell:
$$\text{Ratio} = \frac{\text{Rider App Searches}}{\text{Active Available Drivers}}$$
If $\text{Ratio} > 1.25$, apply exponential multiplier capped at 3.0x to balance local ride requests.

4■■ Atomic Trip Dispatch:
Uses distributed Redis locks (`SETNX trip_lock EX 5`) to prevent duplicate dispatches to the same driver.

■ Have you worked with Geospatial indexing: H3, S2 Geometry, or QuadTrees?

(■ Full live interactive guide in first comment below! ■)
#Uber #Geospatial #Algorithms #SystemDesign #Redis #SoftwareEngineering

--- FIRST COMMENT ---
■ Live interactive masterclass & code: https://mr-elegant.github.io/System-Design-Masterclass/
```

SDI Post #07: Designing Double-Entry Banking Ledgers & Stripe-Style Idempotency

Folder: 02_System_Design_Interview_Flagships_Series/Post_07_Uber_H3_Surge_Pricing/ | **Attach:** animated_diagram.gif

```
■ How Stripe and modern fintech ledgers guarantee zero money creation/loss under network failures:

In financial systems, a single lost penny or duplicate charge can result in catastrophic compliance failure.

Here is how modern banking ledgers maintain 100% mathematical integrity:

1■■ Double-Entry Accounting Invariant:
Money is never created or destroyed; it only moves between accounts.
Every transaction MUST contain balanced Debit and Credit legs:
$$\sum \text{Debits} = \sum \text{Credits}$$

2■■ Idempotency Key Engine:
Client sends header `Idempotency-Key: uuid-v4`.
• API Gateway checks Redis (`SETNX lock:key EX 120`).
• If in-flight, return `409 Conflict` or wait.
• If completed, return cached response directly from Redis without hitting the payment rail!

3■■ Immutable Append-Only Ledger:
Ledger records are NEVER updated or deleted (`UPDATE`/`DELETE` forbidden at SQL grant level). Corrections are applied
via reversing debit/credit journal entries.

4■■ Cryptographic Audit Trail:
Each journal entry stores a SHA-256 hash chaining back to the previous entry: `Hash_N = SHA256(Hash_{N-1} +
EntryData)`.
```

```
■ How do you handle race conditions in payment gateways: Optimistic Locking or Redis Redlock?

(■ Full live interactive guide in first comment below! ■)
#FinTech #Banking #Stripe #Idempotency #SystemDesign #PostgreSQL #SoftwareEngineering

--- FIRST COMMENT ---
■ Live interactive masterclass & code: https://mr-elegant.github.io/System-Design-Masterclass/
```

SDI Post #08: Designing ChatGPT at 100M Scale: Token Streaming & Radix KV-Cache

Folder: 02_System_Design_Interview_Flagships_Series/Post_08_Raft_Consensus_Protocol/ | **Attach:** animated_diagram.gif

```
■ The complete production architecture of ChatGPT serving 100M daily active users:

Serving large language models (LLMs) is fundamentally different from traditional REST APIs because generation is
memory-bandwidth bound and token-by-token sequential.

■ Core System Architecture:

1■■ WebSocket Full-Duplex Gateway:
Maintains persistent bi-directional streams. Allows clients to send mid-stream abort signals (saving expensive GPU
inference tokens).

2■■ RadixAttention (vLLM) KV-Cache Tree:
Maintains a prefix tree of Key-Value attention tensors in GPU VRAM across multiple requests. Reusing shared system
prompts slashes Time-To-First-Token (TTFT) by up to 95%!

3■■ Continuous Iteration-Level Batching:
Traditional batching waits for the slowest request in a batch to complete. Continuous batching evicts completed
sequences and inserts new requests on every token iteration step!

4■■ Compute & Bandwidth Sizing:
At 100M DAU with 50 tokens/sec peak: Requires cluster sizing across 250,000 H100 GPUs with 3.35 TB/s HBM3 memory
bandwidth and 2.56 Gbps network egress.

■ What LLM serving engine do you use: vLLM, TensorRT-LLM, or TGI?

(■ Full live interactive guide in first comment below! ■)
#ArtificialIntelligence #ChatGPT #LLM #MachineLearning #SystemDesign #GenerativeAI

--- FIRST COMMENT ---
■ Live interactive masterclass & code: https://mr-elegant.github.io/System-Design-Masterclass/
```

■ Series 3: Microservices Architecture Patterns (3 Posts)

Folder: LinkedIn_Posting_Vault/03_Microservices_Architecture_Series/

Microservices Post #01: Circuit Breaker Pattern: Preventing Cascading Failures with Resilience4j

Folder: 03_Microservices_Architecture_Series/Post_01_Circuit_Breaker_Pattern/

```
■■ How the Circuit Breaker pattern prevents 1 failing microservice from taking down your entire company:

When downstream Service B slows down or fails, upstream Service A keeps waiting on HTTP timeouts, consuming worker
threads until thread pool exhaustion causes Service A to crash too!

■ The Circuit Breaker State Machine:

1■■ CLOSED (Normal):
Requests flow normally. Tracks success/failure rate.

2■■ OPEN (Tripped):
When failure rate exceeds threshold (e.g. 50% over 20 requests), circuit opens immediately.
All subsequent requests FAIL FAST with cached fallback or default response without calling Service B!

3■■ HALF-OPEN (Testing Recovery):
After a configured timeout (e.g. 10s), allows a trial batch of 5 requests through.
• If they succeed -> Transition back to CLOSED.
• If they fail -> Transition back to OPEN for another backoff period.

(Includes resilience patterns with Exponential Backoff and Full Jitter!)

■ Do you configure Circuit Breakers in code (Resilience4j) or at the Service Mesh layer (Envoy)?
```

```
(■ Full live interactive guide in first comment below! ■)
#Microservices #Resilience #CircuitBreaker #DevOps #SystemDesign #SoftwareEngineering

--- FIRST COMMENT ---
■ Live interactive masterclass & code: https://mr-elegant.github.io/System-Design-Masterclass/
```

Microservices Post #02: Distributed Sagas: Orchestration vs Choreography for Multi-Service Transactions

Folder: 03_Microservices_Architecture_Series/Post_02_Distributed_Sagas/

```
■ Why 2-Phase Commit (2PC) is an anti-pattern in microservices and how Sagas solve distributed transactions:

In microservices, each service owns its private database. A checkout transaction spans Order Service, Payment Service,
and Inventory Service.

■ Why 2PC Fails at Scale:
Two-Phase Commit holds distributed locks across all databases until everyone commits. If 1 service is slow, all
database connections stall, causing latency spikes and blocking throughput.

■ The Solution: The Saga Pattern (A sequence of local transactions).
If a step fails, the Saga executes **Compensating Transactions** in reverse order to undo changes!

■ Choreography (Event-Driven):
• Services publish and listen to domain events over Kafka/RabbitMQ.
• Pros: Simple, no central orchestrator.
• Cons: Hard to track workflow state as service count grows (spaghetti dependencies).

■ Orchestration (Command-Driven):
• A central Saga Orchestrator (Temporal, Camunda, AWS Step Functions) sends explicit commands to each service and
waits for acknowledgments.
• Pros: Centralized visibility, easy rollback tracking, handles complex timeout retries.

■ Do you use Orchestration or Choreography for distributed checkout flows?

(■ Full live interactive guide in first comment below! ■)
#Microservices #SagaPattern #Kafka #DistributedSystems #SoftwareArchitecture #Fintech

--- FIRST COMMENT ---
■ Live interactive masterclass & code: https://mr-elegant.github.io/System-Design-Masterclass/
```

Microservices Post #03: Transactional Outbox Pattern & Debezium Change Data Capture (CDC)

Folder: 03_Microservices_Architecture_Series/Post_03_Transactional_Outbox_Pattern_&_Debe/

```
■ How to update your SQL database and publish to Kafka without dual-write inconsistency:

The Dual-Write Problem:
```python
db.save(order) # Step 1: Succeeds
kafka.publish(order) # Step 2: Fails or network timeout!
```
If Step 2 fails, your DB has the order, but downstream services never get notified! If you swap the order, Kafka gets
the message but the DB write might fail!

■ The Solution: Transactional Outbox Pattern + Debezium CDC.

1■■ Atomic Local Commit:
In a single ACID database transaction, insert the Order into the `Orders` table AND an event record into an `Outbox`
table:
```sql
BEGIN;
INSERT INTO orders (id, user_id, amount) VALUES (101, 42, 99.00);
INSERT INTO outbox (event_id, aggregate_type, payload) VALUES (uuid(), 'ORDER_CREATED', json_payload);
COMMIT;
```

2■■ Change Data Capture (Debezium):
Debezium tails the database Transaction Log (PostgreSQL WAL / MySQL binlog) and streams events to Kafka with
At-Least-Once delivery guarantees and zero application polling overhead!

■ Have you implemented the Outbox pattern with Debezium in production?

(■ Full live interactive guide in first comment below! ■)
#Kafka #EventDriven #PostgreSQL #Debezium #Microservices #SystemDesign
```

```
--- FIRST COMMENT ---
■ Live interactive masterclass & code: https://mr-elegant.github.io/System-Design-Masterclass/
```

■ Series 4: Low Level Design (LLD) with JavaScript (2 Posts)

Folder: LinkedIn_Posting_Vault/04_Low_Level_Design_JavaScript_Series/

LLD JavaScript Post #01: SOLID Principles in Modern JavaScript & TypeScript

Folder: 04_Low_Level_Design_JavaScript_Series/Post_01_SOLID_Principles_in_Modern_JavaScri/

```
■ Mastering the 5 SOLID Principles in Modern JavaScript / TypeScript with real code examples:

Writing clean, maintainable, and extensible code is what separates Junior developers from Senior/Staff engineers.

Here is the quick breakdown of SOLID:

1■■ Single Responsibility (SRP): A class/module should have only ONE reason to change (e.g. separate
`InvoiceCalculator` from `InvoicePDFRenderer`).
2■■ Open/Closed (OCP): Open for extension, closed for modification (use Strategy Pattern to add new discount rules
without editing core pricing logic).
3■■ Liskov Substitution (LSP): Subclasses must be substitutable for their base types without breaking invariants.
4■■ Interface Segregation (ISP): Clients should not depend on interfaces they do not use (split giant interfaces into
focused role contracts).
5■■ Dependency Inversion (DIP): High-level modules should depend on abstractions (interfaces), not concrete
implementations (inject DB repositories via constructors).

(Full JavaScript/TypeScript code examples for all 5 principles in the guide!)

■ Which SOLID principle do you see violated most often in code reviews?

(■ Full live interactive guide in first comment below! ■)
#JavaScript #TypeScript #SOLID #CleanCode #OOP #WebDevelopment #SoftwareEngineering

--- FIRST COMMENT ---
■ Live interactive masterclass & code: https://mr-elegant.github.io/System-Design-Masterclass/
```

LLD JavaScript Post #02: Designing an In-Memory Multi-Tier Rate Limiter in Node.js

Folder: 04_Low_Level_Design_JavaScript_Series/Post_02_Designing_an_In-Memory_Multi-Tier_R/

```
■■ Designing a high-performance Token Bucket & Sliding Window Log Rate Limiter in Node.js:

How do you protect your Node.js APIs from brute-force attacks and noisy neighbors without killing throughput?

■ Two Core Algorithms Compared:

1■■ Token Bucket Algorithm:
• Tokens added at fixed refill rate (e.g., 10 tokens/sec) up to capacity $B$.
• Each request consumes 1 token.
• Strengths: Memory-efficient ($O(1)$ space), allows bursts up to bucket capacity.

2■■ Sliding Window Log Algorithm:
• Tracks exact timestamps of requests in a Redis Sorted Set (`ZSET`).
• Drops entries older than $(now - windowSize)$ and counts remaining elements with `ZCARD`.
• Strengths: 100% boundary accuracy, zero burst leakage at boundary edges.
• Tradeoff: Higher memory footprint ($O(N)$ elements per user).

(Complete executable Node.js class implementation with Lua atomic scripts in the guide!)

■ Do you run rate limiters at the API Gateway level (Envoy/Kong) or as Node.js middleware?

(■ Full live interactive guide in first comment below! ■)
#Nodejs #JavaScript #RateLimiter #Security #Backend #SystemDesign

--- FIRST COMMENT ---
■ Live interactive masterclass & code: https://mr-elegant.github.io/System-Design-Masterclass/
```